Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Computer Science and Engineering (Data Science)

EXPERIMENT NO 1

Name : Joel Dsouza **Roll No : D041** **Sap : 60009230152**

AIM: To study and implement Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)

THEORY:

1. Tokenization:

Tokenization is a common task in Natural Language Processing (NLP). It's a fundamental step in both traditional NLP methods like Count Vectorizer and Advanced Deep Learning-based architectures like Transformers. Tokenization is a way of separating a piece of text into smaller units called tokens. Here, tokens can be either words, characters, or sub words. Hence, tokenization can be broadly classified into 3 types – word, character, and sub word (n-gram characters) tokenization.

For example, consider the sentence: "Never give up".

The most common way of forming tokens is based on space. Assuming space as a delimiter, the tokenization of the sentence results in 3 tokens – **Never-give-up**. As each token is a word, it becomes an example of Word tokenization.

2. Filtration:

Many of the words used in the phrase are insignificant and hold no meaning. For example – English is a subject. Here, 'English' and 'subject' are the most significant words and 'is', 'a' are almost useless. English subject and subject English hold the same meaning even if we remove the insignificant words – ('is', 'a'). Using the nltk, we can remove the insignificant words by looking at their part-of-speech tags. For that, we must decide which Part-Of-Speech tags are significant.

Word	Tag
a	DT
all	PDT
an	DT
and	CC
or	CC
that	WDT
the	DT

3. Stop Word Removal:

All the words in a query are stop words. If all the query terms are removed during stop word processing, then the result set is empty. To ensure that search results are returned, stop word removal is disabled when all the query terms are stop words. For example, if the word *car* is a stop word and you search for *car*, then the search results contain documents that match the word *car*. If you search for *car buick*, the search results contain only documents that match the word *buick*.

The word in a query is preceded by the plus sign (+).The word is part of an exact match.
The word is inside a phrase, for example, "I love my car".

4. Stemming:

Stemming is a part of linguistic studies in morphology and artificial intelligence (AI) information retrieval and extraction. Stemming and AI knowledge extract meaningful information from vast sources like big data or the Internet since additional forms of a word related to a subject may need to be searched to get the best results. Stemming is also a part of queries and Internet engines. Recognizing, searching and retrieving more forms of words returns more results.

Lab Experiments to be Performed in This Session: -

Perform Following Preprocessing Techniques on the given corpus

1. Tokenization,
2. Converting Text Lower Case
3. Remove Numbers
4. Converting Number to Words
5. Remove Punctuation
6. Remove Whitspaces
7. Remove StopWords
8. Count Word Frequency
9. Stemming (Porter Stemmer and Lancaster Stemmer)
10. Lemmatization

```
# ========================================================
# EXPERIMENT NO. 1: TEXT PREPROCESSING USING NLTK
# ========================================================
# AIM: To perform various text preprocessing techniques such as:
# 1. Lowercasing
# 2. Removing numbers
# 3. Converting numbers to words
# 4. Removing stopwords
# 5. Stemming (Porter and Lancaster)
# 6. Lemmatization
# ========================================================

import nltk

import string

import re

# STEP 1: ORIGINAL TEXT
text="There are 3 balls in this bag , and 12 in the other one"
# STEP 2: REMOVE NUMBERS USING REGULAR EXPRESSIONS
re.sub(r'\d+','',text)
# STEP 3: CONVERT TEXT TO LOWERCASE
text2 = text.lower()

import inflect

# STEP 4: CONVERT NUMBERS TO WORDS USING inflect ENGINE
p=inflect.engine()

temp_str = text.split()

new_string = []

for word in temp_str:
  if (word.isdigit()):
    temp = p.number_to_words(word)
    new_string.append(temp)
  else:
    new_string.append(word)

# Join all processed words back into a sentence
answer = ' '.join(new_string)
print(answer)

There are three balls in this bag , and twelve in the other one

from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize
```

```
# STEP 5: REMOVE STOPWORDS
def remove_stopwords(text):
  stop_words = set(stopwords.words("english"))
  word_tokens = word_tokenize(text)
  filtered_text = [word for word in word_tokens if word not in
stop_words]
  return filtered_text

nltk.download('stopwords')
nltk.download('punkt_tab')
remove_stopwords(text)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
['There', '3', 'balls', 'bag', ',', '12', 'one']
```

```
from nltk.stem import PorterStemmer, LancasterStemmer
from nltk.tokenize import word_tokenize


# STEP 6: PORTER STEMMER
stemmer = PorterStemmer()

def stem_word(text):
  word_tokens = word_tokenize(text)
  stem = [stemmer.stem(word) for word in word_tokens]
  return stem

text='yuhijugi efugeu u eduh u eudgu euhd'

print(stem_word(text))
```

```
['yuhijugi', 'efugeu', 'u', 'eduh', 'u', 'eudgu', 'euhd']
```

```
from nltk.stem import PorterStemmer

from nltk.tokenize import word_tokenize

# STEP 7: COMPARISON OF PORTER AND LANCASTER STEMMERS

raw = 'dtghhcgweycgecbw uyeghewb y ydghw u  dged ud eu d'

tokens = word_tokenize(raw)

porter = nltk.PorterStemmer()

Lancaster = nltk.LancasterStemmer()

print([porter.stem(t) for t in tokens])
```

```
['dtghhcgweycgecbw', 'uyeghewb', 'y', 'ydghw', 'u', 'dged', 'ud',
'eu', 'd']
```

```
print([Lancaster.stem(t) for t in tokens])
```

```
['dtghhcgweycgecbw', 'uyeghewb', 'y', 'ydghw', 'u', 'dged', 'ud',
'eu', 'd']
```

```
from nltk.stem import WordNetLemmatizer

from nltk.tokenize import word_tokenize

lemmatizer = WordNetLemmatizer()

# STEP 8: LEMMATIZATION

def lemmaatize_word(text):
  word_tokens = word_tokenize(text)
  lemma = [lemmatizer.lemmatize(word,pos='v') for word in word_tokens]
  return lemma
text = 'uwgdh uwdguqwdgq wdqwhd uugwdhwd'
print(lemmaatize_word(text))
```

```
['uwgdh', 'uwdguqwdgq', 'wdqwhd', 'uugwdhwd']
```

```
# ========================================================
# END OF EXPERIMENT
# ========================================================
# RESULT:
# The experiment demonstrates various preprocessing techniques used in
NLP such as:
# number removal, case conversion, stopword removal, stemming, and
lemmatization.
# These techniques clean and normalize textual data before applying
NLP algorithms.
```